



Softwarearchitektur Projektarbeit

Gen-AI Nachweis

Gruppe 1 - Team 2

Leonid Georg Mikhailovic Stommel

WS 2025/26

Contents

1	Einführung	3
2	Relevante Prompts	4
2.1	Library-Problem bei API-UnitTests	4
2.1.1	Prompting	4
2.1.2	LLM-Ausgabe	4
2.1.3	Lernergebnisse	4

1 Einführung

Die Arbeit mit Künstlicher Intelligenz (KI) und Large Language Models (LLMs) wird immer wichtiger und unumgänglicher. Damit wird es auch immer wichtiger zu lernen wie man verantwortungsvoll mit KI umgeht. Im Optimalfall sollte man nach einem gewissenhaften Prompt in der Softwareentwicklung nicht nur einen schlaueren Code sondern auch einen schlaueren Kopf haben, so dass man es als Hilfsmittel für ein besseres Verständnis für ein Thema einsetzt.

In diesem Dokument dokumentiere ich relevante Prompts, welche mir in der Softwarearchitektur Projektarbeit weitergeholfen haben.

2 Relevante Prompts

2.1 Library-Problem bei API-UnitTests

[Link zum Chat.](#)

2.1.1 Prompting

Die Unit-Tests, welche API-Calls für das Feature "Products" testet, schlugen fehl und ich konnte bei den sehr langen Fehlermeldungen nicht durchblicken woran es lag, da ich dort sehr schnell die Übersicht verlor.

Der LLM habe ich nach Erklärung des Softwarekontextes mein Problem beschrieben und sowohl den Source-Code der Datei `ProductController.java`, als auch die Fehlermeldungen für alle fehlgeschlagenen Tests mitgegeben.

2.1.2 LLM-Ausgabe

Die LLM schlug mir vor, den `JwtTokenProvider` als Mock einzubinden. Doch es gab wiederum eine Fehlermeldung, ebenso lang und unübersichtlich. Diese habe ich dann als Antwort geschickt und den Vorschlag bekommen ebenso noch `JwtConfig` einzubinden. Daraufhin hat es funktioniert.

Abschließend habe ich die LLM noch nach einem Fazit der Problemlösung und was ich daraus gedanklich mitnehmen kann gefragt.

2.1.3 Lernergebnisse

Daraus habe ich gelernt:

- Erkennen von Kausalitäten bei bestimmten Statements, z.B. wenn `@WebMvcTest(ProductController.class)` nur einen abgekapselten Teil lädt, bedeutet dies auch, dass andere Komponenten fehlen, die explizit eingebunden werden müssen, sofern sie für den Test benötigt werden.
- Besseres Verständnis des Mocking-Systems (es werden nicht nur eigene Klassen gemockt, sondern auch weniger offensichtliche Abhängigkeiten).

- Sich nicht von jeder Fehlermeldung abschrecken zu lassen und Fehlerausgaben selbst genauer zu analysieren.